

CODEFEST AD ASTRA 2026

Documento tecnico — Etapa 1: Base de Conocimiento Vectorial

1. El corpus y su preprocesamiento

El corpus entregado por ADL son **1826 documentos** (3 GB) de 21 observatorios repartidos en los tres fenomenos: 760 PDF, 964 JSON, 26 CSV, 73 PBF, 8 imagenes, 6 XLSX y 1 TXT. La heterogeneidad es el primer problema de ingenieria del reto: un mismo indice tiene que absorber informes de 400 paginas de SIPRI o del AI Index, articulos web serializados en JSON, datasets bibliograficos de decenas de MB y teselas de mapas vectoriales. De los 1826 documentos, **1818 aportan texto**; los 8 restantes son 5 imagenes sin texto legible, un JSON vacio y dos archivos con extension *.pdf* que en realidad son paginas HTML de error de la descarga.

Cada formato de origen tiene un extractor dedicado que produce un objeto comun (*RawDocument*) con el texto crudo y metadata auxiliar: **PDF** (pypdfium2, preservando el orden de lectura por pagina y con deteccion heuristica de boilerplate — lineas identicas repetidas en $\geq 30\%$ de las paginas de un documento de 3 o mas paginas se descartan como cabecera/pie de pagina), **HTML** (trafilatura, que separa el contenido principal del menu de navegacion, anuncios y pie de pagina, y preserva encabezados como senales Markdown), **JSON** (mapeo explicito de campos conocidos como *title/body_paragraphs/body_text*, con fallback generico para esquemas no anticipados), **CSV/XLSX** (cada fila se convierte en pares *columna: valor* y se trata como unidad atomica de fragmentacion) e **imagenes** (OCR opcional via pytesseract, con degradacion controlada si el binario de tesseract no esta disponible). Tres decisiones merecen justificacion explicita porque las impuso el corpus real:

- **pypdfium2 en vez de pdfplumber** para los PDF. Con 760 PDF y del orden de 30.000 paginas, el costo de extraccion deja de ser irrelevante: medido sobre este corpus, pypdfium2 lee unas 3.000 paginas en 4,5 s, mientras que pdfplumber es dos ordenes de magnitud mas lento y convierte la fase offline de minutos en horas. Para texto plano de informes la calidad extraida es equivalente.
- **OCR selectivo.** Cerca del 5% de los PDF — casi todos los informes de riesgo de la Defensoria del Pueblo — son escaneos sin capa de texto. Cuando un documento rinde menos de 200 caracteres por pagina se asume escaneo y se rasteriza para pasarlo por tesseract, con un tope de 60 paginas por documento: sin ese tope un puñado de escaneos largos domina el tiempo total de la corrida.
- **Los .pbf no son OpenStreetMap.** Las 73 teselas de Amazon Underworld son *Mapbox Vector Tiles*, no el formato Protocol Buffer de OSM, asi que las librerias habituales (pyosmium, pyosm) no las leen. Se decodifican con *mapbox-vector-tile*, se recorren las capas leyendo los atributos de cada elemento como pares *atributo: valor*, y se deduplican los elementos repetidos entre niveles de zoom, tal como indica la sec. 2.1.

Los CSV/XLSX llevan un tope de 500 filas por archivo. Los datasets del AI Index son volcados bibliograficos de PubMed de hasta 35 MB, ajenos a las consultas del reto: sin tope aportarian mas de 100.000 fragmentos de ruido que dominarian el espacio vectorial. Con tope, el documento sigue indexado y puede recuperarse a nivel documento, pero no ahoga al resto del corpus. La limpieza posterior (normalizacion Unicode NFC, remocion de caracteres de control, deteccion de idioma) es deliberadamente conservadora sobre el contenido — sin lowercasing ni stemming — porque la evaluacion compara el campo *text* de forma textual contra el ground truth.

Reparacion del guion de fin de linea. Las fuentes incrustadas de muchos PDF no mapean el guion de corte a Unicode y el extractor lo entrega como U+FFFE, partiendo la palabra: *sate■lites*, *weap■ons*. Afectaba al **30% de los fragmentos** (38.397 de 128.526, en 613 documentos) y degradaba las dos metricas a la vez: rompe para el tokenizer justo los terminos de dominio que discriminan la consulta, y entrega al evaluador un fragmento con el texto partido. La reparacion decide entre unir (*satellites*) y conservar el guion (*AI-related*) contando en el propio corpus cual de las dos formas aparece mas veces —

23.177 cortes quedaron resueltos con esa evidencia — y solo cae en una heurística cuando ninguna forma está atestiguada. Medido sobre el ground truth propio, la reparación no cambia F1@3 de forma significativa (0.306 a 0.310; gana 3 consultas, pierde 3, empata 35): se adopta porque corrige un defecto del dato y limpia los 166 fragmentos entregados que salían partidos, no por una ganancia de recuperación que la muestra no puede sostener.

1.1 Identificación de documentos (doc_id)

El emparejamiento a nivel documento se hace con el **DOC_ID** que suministra ADL en el inventario del corpus (formato *F1-AIINDEX-001*), no con un identificador propio. El manifest se indexa por **ruta relativa** y no por nombre de archivo: 59 nombres del inventario aparecen en dos carpetas distintas con DOC_ID distintos — los informes de CSET bajo *pdfs/Reports* y *pdfs/Translation*, y las teselas que se repiten por nivel de zoom — de modo que indexar por nombre le habría asignado el identificador equivocado a 127 documentos y anulado su aporte al F1@3. Once archivos presentes en el corpus no figuran en el inventario de ADL (catálogos y registros del proceso de descarga); al no tener DOC_ID no pueden aparecer en el ground truth, y se excluyen del índice para que no ocupen uno de los tres cupos de documento por consulta.

2. Estrategia de chunking y justificación

Se implementó una estrategia **híbrida estructural + oracional con solapamiento**, en tres niveles:

- **Estructural:** el texto se separa primero por encabezados Markdown (que HTML/JSON/MD ya traen marcados por el extractor); si el documento no tiene encabezados (caso típico de PDF), se trata como una única sección.
- **Segmentación de oraciones multilingüe:** dentro de cada sección, se usa *pysbd* (reglas lingüísticas dedicadas) para español e inglés, y el parser entrenado de spaCy (*pt_core_news_sm*) para portugués, que *pysbd* no cubre nativamente.
- **Empaquetado voraz por presupuesto de tokens** (~280 tokens, margen bajo el límite de 512 del encoder), contando tokens con el tokenizer real del encoder de indexación — no una aproximación por palabras — con **solapamiento de 1 oración** entre chunks consecutivos para no perder ideas que caen en la frontera.

La unidad mínima que se mueve entre chunks es siempre una oración completa: el empaquetador nunca corta un carácter a mitad de oración, cumpliendo el requisito de completitud lingüística (sec. 3.3). Esta garantía se verifica automáticamente en *tests/test_chunking.py*. La única excepción estructural es CSV/XLSX, donde cada fila (no una oración en lenguaje natural) se indexa como chunk atómico, tal como permite explícitamente la especificación.

3. Encoder(s) seleccionado(s) y criterios de elección

Encoder primario: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** (arquitectura encoder tipo BERT, licencia Apache 2.0).

Criterio	Justificación
Soporte multilingüe	Cubre nativamente español, inglés y portugués (50+ idiomas), necesario porque el corpus y las consultas mezclan los tres.
Dimensionalidad	384 dimensiones: balance razonable entre costo de almacenamiento/busqueda y expresividad para el volumen esperado del reto.
Longitud máxima de entrada	512 tokens; el presupuesto de chunking (~280 tokens) deja margen suficiente.
Licencia	Apache 2.0, uso libre sin restricciones para el contexto del reto.

Eficiencia computacional	Modelo liviano (backbone MiniLM), adecuado para los recursos de computo limitados del equipo.
Riesgo de reproducibilidad	Ya validado en el material de apoyo entregado por el equipo organizador (01_embeddings.ipynb, 02_rag.ipynb), reduciendo el riesgo de incompatibilidades de ultimo momento.

3.1 Encoder secundario y fusion de rankings (sec. 4.4)

Actualizacion: la entrega usa una cascada de **tres** encoders: MiniLM trae los candidatos y los re-puntuan **gte-multilingual-base** (encoder-only, Apache 2.0, 305 M, 768 dim) y E5, con peso 0,25 cada uno. Elegida midiendo cinco estructuras: NDCG@10 sube de **0,338 a 0,406** (41 consultas) y de 0,329 a 0,360 (10 independientes). GTE como *primario* se descarto pese a su mejor F1 en las 41 (0,385) porque cae a **0,200** en las independientes: sesgo de pooling.

El pipeline soporta construir la base con mas de un encoder. El secundario es **intfloat/multilingual-e5-base** (encoder tipo BERT, licencia MIT, 768 dimensiones, multilingue nativo, limite 512 tokens). El criterio de seleccion no fue elegir un modelo "mejor" en abstracto, sino uno con **errores decorrelacionados** respecto al primario: E5 esta entrenado para recuperacion densa mientras que el primario esta afinado para similitud de parafrasis. Fusionar dos rankings solo aporta cuando los modelos se equivocan en casos distintos; dos encoders de la misma familia habrian aportado poco.

La familia E5 exige prefijos distintos para consulta (*query:*) y pasaje (*passage:*); omitirlos degrada la calidad silenciosamente. Por eso la interfaz *Encoder* distingue codificacion asimetrica (*encode_query / encode_passages*), y los encoders que no los requieren heredan el comportamiento por defecto.

Las listas se combinan con **Reciprocal Rank Fusion** (RRF, ecuacion 7, con $k_0=60$), elegido sobre CombSUM/CombMNZ porque opera sobre posiciones y no sobre puntuaciones absolutas: es robusto a que dos encoders produzcan similitudes en escalas distintas. El grafo de conocimiento se incorpora como una lista adicional a fusionar, tal como sugiere la sec. 8.5.

Invariante de correccion: la fusion empareja fragmentos por *chunk_id*, lo que solo es valido si todos los indices comparten los mismos fragmentos. Como el presupuesto de chunking depende del tokenizer, fragmentar por separado con cada encoder produciria *chunk_id* colisionantes apuntando a textos distintos. Por eso el chunking se ejecuta una unica vez y sus fragmentos se reutilizan para todos los encoders; *generador.py* ademas verifica la coincidencia al cargar y aborta si detecta indices desincronizados, en lugar de producir resultados silenciosamente incorrectos.

Resultado de la medicion: la fusion simetrica no sirve, la cascada si. Comparadas contra el primario solo (F1@3 0,306 sobre las 41 consultas anotadas), *multilingual-e5-base* en solitario rinde 0,182 y pierde 7-17, la fusion RRF de ambos rinde 0,268 y pierde 8-13, y la **cascada 0,352 ganando 5-0 sin perder ninguna**. Sobre las 10 consultas de anotacion independiente el reparto es el mismo en direccion.

Re-medicion tras reparar los guiones (seccion 1). Reconstruidos los indices sobre el texto reparado la comparacion se sostiene: el primario solo pasa a **0,310 y 0,300**, y la cascada a **0,344 y 0,333**, ganando **4-0 y 1-0 sin perder ninguna**. La prueba de signos no alcanza significancia ($p = 0,125$); se conserva la cascada por el criterio fijado antes de medir, no por el promedio.

Antes de nada se verifico que el segundo encoder no estuviera mal empleado, porque la familia E5 degrada en silencio si se omiten sus prefijos: la indexacion aplica *passage:* y la consulta *query:*, tanto en el pipeline como en la copia autocontenida. Su bajo rendimiento en solitario es real, no un error de uso — y esa es justamente la clave del diseno.

Por que la fusion simetrica no podia funcionar. RRF premia el *acuerdo* entre listas. Medido sobre las 50 consultas, los dos encoders comparten apenas el **11,3%** de los documentos del top-3 y el **6,2%** de los fragmentos del top-10. Con ese nivel de desacuerdo RRF no fusiona: intercala la lista buena con la mala, y el resultado queda a medio camino entre ambas. Eso es exactamente lo observado (0,306 baja a 0,268).

La cascada, que es lo que se entrega. Lo que E5 hace mal es el *recall*: su propio conjunto de candidatos rara vez contiene el documento correcto. Pero puntuar candidatos que el primario ya encontro es un trabajo distinto y mas facil. Asi que el primario genera 200 candidatos y E5 solo los **re-puntua**, sumando su similitud a la del primario con un peso de 0,25. Los dos terminos son cosenos, misma escala, de modo que sumarlos es legitimo. La operacion es barata porque los dos indices se construyen sobre los mismos fragmentos en el mismo orden: el vector de un fragmento en el segundo espacio se lee por su fila con *reconstruct*, sin volver a codificar ni un pasaje. El costo por consulta es una sola vectorizacion adicional.

El peso 0,25 no es el que maximiza el promedio: con 0,5 y 1,0 el F1@3 sobre las 41 consultas sube algo mas (0,354 y 0,359), pero esas variantes empiezan a *perder* consultas en la muestra de anotacion independiente, que es la senal clasica de sobreajuste. Con 0,25 la cascada **no empeora ninguna consulta de ninguna de las dos muestras**: gana 5 y pierde 0 sobre las 41, y da exactamente el mismo resultado que el primario sobre las 10 independientes. Se prefirio la variante que nunca hace dano sobre la que promedia mejor. La prueba de signos sobre las 5 consultas que difieren da $p = 0,062$: es el efecto mejor sustentado que se ha medido en este proyecto, aunque con 41 consultas siga sin cruzar el umbral convencional del 5%.

No se emplea ningun modelo decoder/generativo en ninguna etapa de indexacion o recuperacion (sec. 4.2 y 8.3): en particular, se descarto explicitamente HyDE (requiere un LLM para generar una respuesta hipotetica) y el reranking con LLM. El uso de un *cross-encoder* (arquitectura encoder, no generativa, demostrado en el material de apoyo) quedo implementado detras de un flag desactivado por defecto, como exploracion documentada, para no depender de el en el flujo evaluado.

4. Indice FAISS

Se utiliza **IndexFlatIP** sobre vectores normalizados a norma unitaria, equivalente a similitud coseno (sec. 5.2 y 8.2). Es la opcion recomendada por la propia especificacion para el volumen de documentos esperado en esta etapa del reto: garantiza resultados exactos (sin la perdida de exactitud de IVFFlat/HNSW) y su costo computacional es totalmente asumible a esta escala. El identificador interno de FAISS se mantiene alineado linea a linea con *metadata.jsonl* (mismo orden de insercion que de escritura), invariante verificado en *tests/test_faiss_alignment.py*. Si el corpus real de ADL resultara mucho mayor al esperado, *src/config.py* centraliza el punto donde migrar a un indice aproximado sin tocar el resto del pipeline.

La decision se verifico empiricamente antes de fijarla. Con vectores de 384 dimensiones y $k=120$ sobre CPU, una busqueda exacta tarda 1,9 ms (mediana) con 20.000 vectores, 5,7 ms con 50.000 (9,8 ms en el percentil 99) y 13,2 ms con 100.000. El corpus real produce **128.526 fragmentos**, es decir del orden de 15 ms por consulta: irrelevante frente a los ~27 ms que cuesta vectorizar la consulta y los ~19 s de carga del encoder. Migrar a IVFFlat o HNSW habria cambiado busqueda exacta por busqueda aproximada — perdiendo recall, que es justamente lo que mide la evaluacion — a cambio de ahorrar milisegundos que nadie percibe.

5. Grafo de conocimiento (componente bonus)

NER: se usa el componente de reconocimiento de entidades ya entrenado en los modelos spaCy *es/en/pt_core_news_sm* (licencia MIT, arquitectura no generativa), filtrando tipos de entidad puramente numericos/temporales (fechas, cantidades, porcentajes) para quedarse con personas, organizaciones, lugares y otras entidades nombradas relevantes.

Extraccion de relaciones: heuristica basada en dependencias sintacticas (permitida explicitamente por la especificacion como alternativa a un modelo entrenado de RE): para cada oracion con al menos dos entidades, se toma el verbo/auxiliar entre un par de entidades consecutivas como etiqueta de relacion (o el verbo raiz de la oracion si no hay uno intermedio), con una relacion generica de co-ocurrencia como ultimo recurso.

Construccion e integracion: las tripletas se acumulan en un *networkx.MultiDiGraph*, cada arista con *doc_id*, *chunk_id* y la relacion como evidencia trazable, exportado a GraphML. En recuperacion, las mismas entidades se detectan en la consulta, se buscan sus vecinos de primer orden en el grafo, y los chunks vinculados se fusionan con el ranking vectorial mediante Reciprocal Rank Fusion (el grafo se trata como un indice adicional, sec. 8.5), detras de un flag *--use-graph* opcional en *generador.py*.

El grafo sobre el corpus real tiene 224.101 nodos y 754.876 aristas, construidas a partir de 1.687 documentos. Dos decisiones lo hicieron utilizable. La primera: se excluyen los formatos sin narrativa (CSV, XLSX y las teselas vectoriales, el 17,7% de los fragmentos), porque el NER busca entidades nombradas en lenguaje natural y aplicado a una fila de tabla produce ruido — las entidades mas frecuentes de una version preliminar eran *FALSO*, *VERDADEIRO* y el nombre de una capa de mapa. La segunda: los nombres de entidad se limpian de caracteres de control, que el texto de OCR arrastra y que hacen fallar la exportacion a GraphML, un formato XML que no los admite.

El grafo se entrega, pero los resultados se generan sin el. Medida su fusion con el ranking vectorial contra el ground truth propio, no mejora en ninguna consulta de las 10 de anotacion independiente (pierde 3-0) y gana solo 1 de 41 en el conjunto completo (pierde 7-1). Se entrega por tanto el artefacto, que es lo que pide la seccion 7, con la recuperacion en su configuracion puramente vectorial. Que el componente bonus exista y sea trazable al corpus no obliga a degradar la metrica usandolo.

6. Modulo de recuperacion

- Mismo encoder para indexacion y consulta; vector de consulta normalizado igual que el indice.
- Sobre-recuperacion de candidatos antes de aplicar post-filtros de metadata (fenomeno, formato, idioma) o de umbral de similitud coseno (sec. 8.7).
- Agregacion a nivel documento por **suma** de las puntuaciones de sus fragmentos, sumando los **5 mejores** de cada uno, sobre un pool de **100 candidatos** mas amplio que los 10 fragmentos mostrados, para que la relevancia de un documento no dependa solo de si su mejor chunk aparece en el top-10 (sec. 8.6). La eleccion de la suma sobre el maximo esta medida, no supuesta: ver seccion 7.
- Reciprocal Rank Fusion ($k_0=60$) como mecanismo unico de combinacion, reusado tanto para multiples encoders como para vector+grafo (sec. 8.4).
- Limite de 250 palabras por fragmento aplicado dividiendo en limites oracionales completos cuando un chunk lo excede; los sub-fragmentos comparten *chunk_id* y ocupan su propio *rank* (sec. 9.2.1). El corpus real obligo a agregar un ultimo recurso: en texto proveniente de OCR la puntuacion puede desaparecer por completo y el segmentador devuelve una unica "oracion" de mas de 250 palabras, que respetar el limite oracional dejaria pasar entera. En ese caso — y solo en ese — el corte se hace por palabras.
- **Supresion de fragmentos con texto repetido.** El corpus contiene documentos duplicados (el mismo informe de CSIS bajo dos *doc_id*, series que reeditan capitulos completos), de modo que dos fragmentos distintos del indice pueden tener texto identico. Entregar el mismo texto dos veces no puede aportar ganancia en $NDCG@10$ — el ranking ideal no lo contiene — y ademas desplaza fuera del top-10 a un candidato que si podria aportarla. Sobre las 50 consultas oficiales, 17 de los 500 fragmentos entregados eran duplicados exactos; suprimirlos y completar el cupo con el siguiente candidato recupera esos 17 espacios. La comparacion es de texto exacto (normalizando solo espacios y mayusculas) y no de similitud aproximada, que podria descartar fragmentos legitimamente distintos que comparten un parrafo.
- **Los fragmentos se ordenan despues de decidir los documentos**, con tres criterios de desempate y sin descartar nada, para que las dos mitades de la respuesta no se contradigan: primero los fragmentos de los 3 documentos entregados; luego los que el evaluador puede leer (el corpus trae traducciones al coreano, ruso, arabe, chino y aleman que competian por los mismos cupos); por ultimo los que son aparato bibliografico, que la sec. 10.2.1 juzga por su texto y puntuan 0 aunque su documento sea el

correcto. El primer criterio es la mejora individual mas grande del proyecto: los fragmentos procedentes del top-3 pasan del **32% al 98%** y el NDCG@10 sube **+0,131** (IC 90% [+0,066, +0,199]) sin mover el F1@3, como corresponde a un cambio que no toca los documentos.

Ninguna etapa de recuperacion usa un modelo generativo: solo vectores, puntuaciones de similitud coseno y aritmetica sobre metadata.

7. Como se tomaron las decisiones: medicion interna

El ground truth oficial no es publico durante el reto, de modo que ninguna decision de diseno puede validarse contra la metrica real. Para no elegir a ojo se construyo uno propio (*dev/eval/*) que cubre **las 50 consultas oficiales**: 41 con documentos relevantes marcados y 9 en las que se reviso candidato por candidato sin que ninguno respondiera. Se construyo en dos etapas y la distincion es importante para no enganarse:

- **10 consultas de anotacion independiente.** Los candidatos salieron de contar palabras clave sobre el texto extraido, *sin pasar por el recuperador*. Son las unicas validas para comparar dos encoders entre si.
- **31 consultas por pooling** (la tecnica de TREC): los candidatos los propuso el propio sistema. Aportan volumen, pero una consulta cuyos candidatos propuso el encoder X favorece a X, porque un documento que X nunca recupero no pudo marcarse. Por eso cada linea registra su procedencia y la evaluacion tiene una opcion *--sin-pooling*.

El sesgo resulto menor de lo temido: el F1@3 da 0,306 sobre las 41 consultas y 0,300 sobre las 10 independientes. La razon es que los candidatos se proponen con un pool de 200 mientras la configuracion entregada usa 60, de modo que las marcas no se cumplen solas.

La agregacion a documento **suma** las puntuaciones de sus fragmentos en lugar de tomar el **maximo**, y con el pool ampliado a 100 suma solo los 5 mejores. La razon principal no es el promedio (0,306 frente a 0,226 sobre las 41 anotadas, pero contando por consulta 16-8 con 17 empates, **p = 0,15**: no alcanza significancia) sino el argumento estructural: un documento verdaderamente relevante contiene *varios* pasajes relevantes, mientras que el maximo premia al que tuvo un unico fragmento afortunado. Se documenta asi y no como resultado medido: la diferencia entre "lo medimos" y "lo argumentamos y el dato no lo contradice" es lo que evita sobreajustar a un ground truth reducido.

El promedio de F1@3 no basta para decidir. El tamano del pool parecia importar: con 26 consultas, un pool de 30 daba 0,309 frente a 0,274 con 60, y la ventaja se repetia con 10, 19 y 26 consultas. Contando por consulta el resultado es otro: **30 gana en 5, 60 en 3 y empatan 18**, que es lo que produce el azar. Con unas 30 consultas cada una pesa 0,033 en la media, asi que dos que cambien de lado la mueven mas que cualquier efecto real. El pool quedo en 60 hasta que una medicion posterior, con criterio de intervalo de confianza y no de promedio, justifico ampliarlo a 100.

El mismo criterio descarto otras dos hipotesis que parecian prometedoras: que fallara la recuperacion entre idiomas distintos (2 de 5 aciertos con documentos en ingles frente a 3 de 5 en espanol) y que el sistema confundiera los grupos armados ilegales con las fuerzas armadas estatales (medido con un patron justo, afecta al 8% de los cupos). Se documentan porque el riesgo de un ground truth reducido no es medir de menos sino *sobreajustar*.

Donde esta el cuello de botella. Antes de seguir ajustando parametros se midio que impide acertar cuando el sistema falla, separando los tres motivos de un F1@3 nulo: documento no indexado, ningun fragmento suyo en el pool, o que entre y pierda la agregacion. De las 17 consultas en cero, ninguna por indexacion, dos porque el encoder no alcanza el documento y **quince con el documento correcto dentro del pool**. El limite no esta en la recuperacion sino en la agregacion.

De ese diagnostico salieron dos intentos, ambos implementados y descartados por la misma regla. El primero, sumar solo los *M* mejores fragmentos de cada documento: la suma sin tope deja que un

documento con muchos fragmentos mediocres desplace a uno con un fragmento excelente. El promedio mejora (0,347 con pool 100), pero es el maximo de setenta combinaciones y por consulta reparte 8-3 con 30 empates ($p = 0,227$). El segundo, BM25 fusionado con el denso por RRF para rescatar los terminos raros que un vector de 384 dimensiones diluye: rinde 0,192 frente a 0,306 y pierde 15-4 ($p = 0,019$). La hipotesis lexica queda refutada como mejora general.

El codigo de las dos queda en el repositorio con su medicion documentada. El contraste con la cascada da valor al criterio: el maximo de la grilla de agregacion (0,347) es comparable al de la cascada (0,352), pero una sobrevive al conteo por consulta y las otras reparten victorias como lo haria el azar.

Estado final de la medicion. Sobre las 50 consultas la entrega da $F1@3 = 0,402$ y $NDCG@10 = 0,457$. El $F1@3$ no tiende a 1: la sec. 10.2.2 fija $P@3 = \text{aciertos}/3$ con los tres cupos siempre llenos, asi que una consulta con un solo documento relevante topa en 0,50 y el techo sobre este ground truth es **0,906** — el 0,402 es el 44% de lo alcanzable, no el 40% de 1.

Las dos ultimas mejoras adoptadas se componen. La primera es **ampliar el pool de 60 a 100**: la cascada recuperaba 200 candidatos y se descartaban 140 antes de agregar, de modo que la profundidad extra solo reordenaba. La segunda es un **glosario bilingüe** que expande la consulta antes de vectorizarla, justificado por una propiedad medible del corpus: las consultas estan en espanol y los fenomenos 1 y 2 en ingles, y *todos* los terminos de dominio que las consultas usan en espanol son entre 30 y 2.000 veces mas raros que su forma inglesa (NBQR 0 veces contra 66 de CBRN; "antisatelite" 8 contra 3.813 de ASAT). Es una tabla escrita a mano, sin modelo generativo ni traductor (sec. 8.3); una consulta sin terminos de la tabla se devuelve identica.

Medidas sobre el ground truth propio, cada una por separado y las dos juntas ($F1@3$ y $NDCG@10$, primero sobre las 50 consultas y luego sobre las 41 de anotacion humana): la configuracion anterior daba **0,363 / 0,392 y 0,386 / 0,405**; solo el glosario, 0,379 / 0,403 y 0,396 / 0,406; solo el pool ampliado, 0,366 / 0,434 y 0,398 / 0,447; y **las dos juntas, que son las que se entregan, 0,402 / 0,457 y 0,424 / 0,456**. Los efectos se componen: ninguna de las dos anula a la otra.

Con el pool ampliado la agregacion pasa de *suma* a **sumar los 5 mejores fragmentos** de cada documento, por robustez y no por promedio: con pool 60 las dos son *identicas* (ningun documento aporta mas de 5 fragmentos a un pool tan chico), pero al ampliarlo la suma deja de tener tope y un documento con muchos fragmentos mediocres desplaza al bueno — con pool 200 la suma cae a **0,298** mientras el tope de 5 sube a 0,406. Por eso el pool quedo en 100 y no en 200.

Lo que cuestan, dicho explicitamente: sobre las 10 consultas de anotacion independiente el $F1@3$ baja de 0,333 a 0,300 y el $NDCG@10$ de 0,360 a 0,338. Es *una sola consulta* la que cambia de lado y el intervalo de confianza con esa muestra no distingue la perdida de cero, pero es la unica muestra sin sesgo de pooling y corresponde decir que ahi no confirman. Se adoptan porque ganan de forma consistente en las 41 de anotacion humana — $F1@3 +0,038$ (IC 90% [+0,002, +0,073]) y $NDCG@10 +0,051$ ([+0,008, +0,095]) — y porque las dos tienen explicacion mecanica previa a la medicion. Otros dos ajustes se midieron y *no* se adoptaron: reservar cupos de fragmento degrada el $NDCG@10$ de forma monotona (−0,025 con 8 cupos, −0,082 con 4) sin mover el $F1@3$, y el prior de recencia resulta inerte; quedan implementados y apagados. El grafo, re-medido con $NDCG@10$, pierde 11-0: se entrega como artefacto bonus, no en la recuperacion.

La entrega se verifica de punta a punta antes de empaquetarse con *scripts/validar_entrega.py*, que comprueba la estructura de carpetas, las 50 lineas, los 3 documentos y 10 fragmentos por consulta, el limite de 250 palabras, la alineacion entre el indice FAISS y *metadata.jsonl*, y que todo *doc_id* reportado pertenezca al inventario de ADL. La suite de pruebas (126 casos) cubre los invariantes criticos, incluida la ejecucion de *generador.py* como subprocesso con *PYTHONPATH* vacio para garantizar que es autocontenido y que los resultados son reproducibles, requisito del punto 4 de la sec. 1.4.

8. Limitaciones conocidas y decisiones documentadas

- **El ground truth propio es la limitacion dominante del proyecto.** Los valores de F1@3 de la seccion 7 sirven para ordenar configuraciones entre si, no para estimar la nota: la anotacion es parcial frente a un corpus de 1826 documentos, así que un documento relevante no anotado cuenta como error y el valor real sera mayor. Además se juzga el documento viendo un solo fragmento — el mejor de cada documento, que no siempre es la mejor evidencia — lo que hace marcar de menos, nunca de mas.
- **El NDCG@10 se mide con un proxy:** la relevancia de cada fragmento se hereda de su documento, porque anotarla de verdad exigiria relevancia graduada fragmento por fragmento. El proxy sobreestima — da 1 a la bibliografia de un documento relevante — y cuanto, esta acotado por abajo: descontando el aparato bibliografico da 0,443 frente a 0,457. El otro modo de fallo, un pasaje que no responde dentro de un documento que si es relevante, no lo ve ninguna medicion automatica.
- **Documentos sin texto recuperable:** 8 de los 1826. Cinco son imagenes (una de ellas en formato AVIF, que Pillow no lee sin plugin adicional), una es un JSON de 0 bytes y dos son paginas HTML de error guardadas con extension .pdf. Estos ultimos se detectan por contenido y se procesan como HTML, pero su texto es una pagina de error y no aporta nada. Estos documentos quedan en el corpus sin posibilidad de ser recuperados.
- **Campo fuente:** se reporta siempre el nombre de archivo estandarizado del inventario de ADL, nunca la URL de origen (que se conserva aparte, en un campo *url* adicional). La sec. 10.2.1 sugeria *fuentes* como clave de emparejamiento y la aclaracion posterior de ADL indico *doc_id*; reportar el nombre de archivo hace que el sistema funcione bajo cualquiera de las dos lecturas.
- **Fusion de chunks cortos adyacentes** (sec. 9.2.1, mencionada como posible, no obligatoria) no esta implementada: solo se implemento la division obligatoria de chunks que exceden 250 palabras.
- **Boilerplate de PDF:** la heuristica de lineas repetidas ($\geq 30\%$ de paginas) requiere documentos de 3 o mas paginas; PDFs mas cortos no reciben este filtro.